

SCAtt: Synthetic Community-Aware Attributed Graph Generator for Graph Neural Network Benchmarking

Guilherme Henrique Messias^a, Mariana Caravanti de Souza^b, Sylvia Iasulaitis^c, Alan Demétrius Baria Valejo^a

^a*Department of Computing, Federal University of São Carlos,*

^b*Faculty of Computing, Federal University of Mato Grosso do Sul,*

^c*Department of Social Science, Federal University of São Carlos,*

Abstract

Graph Neural Networks (GNNs) are powerful models for handling attributed graphs in tasks such as classification, link prediction, and community detection. However, progress in community detection is hindered by the lack of high-quality datasets, since ground-truth community labels are often unavailable, and the majority of proposed algorithms in recent literature uses the same benchmark datasets to train the models. To address this issues, attributed random graph generators are commonly employed to create synthetic graphs for assessing the strengths and limitations of GNN models. Yet, most generators rely heavily on power-law degree distributions, despite recent evidence showing that scale-free networks are rare, particularly in social network contexts. Moreover, state-of-the-art attributed graph generators provide limited flexibility, as they do not allow users to construct communities with varying densities, degree distributions and sub-community structure. To overcome these limitations, we introduce the Synthetic Community-Aware Attributed Graph Generator (SCAtt), a graph generator algorithm that allows users to control the node degree distribution and sub-community structure. We evaluate SCAtt over tree different tasks, such as graph mimicking, hyper-parameter evaluation and node clustering tuning. The results show that our model overcome the state-of-the art models in synthetic graph generation and data augmentation.

Keywords:

Synthetic Graphs, Graph Neural Networks, Community Detection, Clustering Evaluation.

1. Introduction

Community detection in attributed graphs is a fundamental task in network science and machine learning, with applications ranging from social network analysis [1] to biological systems [2]. The evaluation of algorithms in this domain, however, is highly dependent on the availability of suitable datasets, and the performance of state-of-the-art models not yet reached a stage where the results are equal to ground truth partitions [3]. Furthermore, community detection requires this ground-truth information to reflect group structures, yet such labeled datasets are scarce and often difficult to obtain [4]. This scarcity has motivated the research community to rely on datasets originally designed for classification tasks as a proxy for evaluating clustering and community detection methods [5].

Although convenient, the use of classification datasets for this purpose is problematic. Many of these datasets do not exhibit inherent community-like structures, which undermines their suitability for assessing algorithms aimed at uncovering cohesive groups in networks [5]. These limitations highlight the pressing need for synthetic attributed graph generators that better capture the properties required to evaluate community detection approaches [6].

To overcome these challenges, several algorithms have been developed to generate synthetic graphs tailored for evaluation. Some approaches rely on latent factors to construct networks [7], while others employ traditional models such as the Barabási–Albert [8] model or preferential attachment [9] to reproduce well-known structural properties. In addition, certain methods enable the mimicry of existing datasets, thereby functioning as data augmentation strategies for training Graph Neural Networks (GNNs) [10, 4]. These efforts have contributed to alleviating the scarcity of benchmark data, but they still face significant limitations, particularly in their inability to flexibly generate datasets where each community can exhibit distinct densities, vertex counts, and edge distributions. Moreover, when mimicry is applied, most approaches struggle to preserve the original node degree distribution, which undermines their capacity to reproduce realistic structural patterns necessary for robust evaluation of community detection algorithms.

To address these limitations, we propose a Synthetic Community-Aware Attributed Graph Generator (SCAtt) model. The framework is composed of four main stages: Node community and sub-community assignment, Edge construction, Noise generation and Attribute generation stages. With this, the proposed model is capable of (1) generating artificial datasets according

to user-specified distributions, (2) mimicking existing datasets while preserving their structural and attribute characteristics and (3) Augment a given mimicked graph. To assess these capabilities, we evaluate SCatt in three tasks: Parametric evaluation in synthetic data generation, scale-free analysis over mimicked data according to [11] and [12] and parametric evaluation of a cluster algorithm of the literature.

Our main contributions can be summarized as follows:

- We introduce a novel model for artificial attributed graph generation based on sub-community structures. This model allows the creation of graphs with flexible community-level characteristics, such as varying densities and distinct intra-cluster edge distributions.
- A model that is also capable of dataset mimicking, maintaining local structures of sub-communities and being able to augment the original graph, also serving as a graph augmentation algorithm.
- We conduct a comprehensive evaluation of state-of-the-art graph generation models, considering both the generation of synthetic datasets for benchmarking community detection with GNN-based approaches and the feasibility of dataset mimicking while maintaining degree distributions. This dual evaluation highlights the advantages of SCatt in producing more realistic and diverse attributed graphs for assessing community detection algorithms.

The remainder of this paper is organized as follows. Section 2 presents the problem statement and reviews related work on synthetic attributed graph generation. Section 3 describes the proposed SCatt model and its generation mechanisms. Section 4 details the experimental setup and empirical evaluation. Finally, Section 5 discusses the main findings and concludes the paper.

2. Problem Statement and Related Works

In this section we present the problem statement and the related works in synthetic graph generation.

2.1. Problem Statement

Synthetic attributed graph generation addresses the problem of constructing graphs with both structural and attribute information under different modeling assumptions. In general, existing approaches can be grouped into two broad problem settings. In the first setting, the goal is to generate a graph from a set of user-specified constraints, such as the number of nodes and edges, degree distributions, or attribute assignment mechanisms. In the second setting, the objective is to generate a synthetic graph that resembles a given real graph, preserving selected structural and attribute-related properties.

Formally, in the first setting, given a set of user inputs $\mathcal{S}$, including, for instance, the number of nodes, the number of edges, degree distributions, and probability assignment matrices, the goal is to generate a graph $G = (V, E, X, Y)$ that satisfies the specified constraints. In the second setting, given an input graph $G = (V, E, X, Y)$, the objective is to construct a synthetic graph $\bar{G} = (\bar{V}, \bar{E}, \bar{X}, \bar{Y})$ that preserves relevant structural and attribute-related properties of the original graph.

2.2. State-of-the-art in Attributed Graph Generators

DANCer [9] is a dynamic attributed network with community structure generator, that leans on proprieties of real-world networks such preferential attachment, small world and homophily. The network generation consists of two phases: (1) Built a initial graph with the well-know network proprieties, and (2) modify the initial graph with micro (removing or adding vertices and edges) and macro operations (mitigating members of a community to a new community or existing one, splitting communities and merging then).

GenCAT [4] generates graphs from user-specified inputs that control the relationship between node attributes and labels, enabling the modeling of both core and border nodes, as well as homophilic and heterophilic interaction patterns. The framework is scalable and flexible in terms of structural and semantic configurations. However, despite its usefulness, GenCAT restricts the node degree distribution to power-law regimes, limiting its ability to generate graphs following alternative degree distributions.

The node degree distribution (NDD) encodes both local and global structural properties of a graph, providing essential insights into its connectivity patterns and overall organization. In this context, [13] proposed a computationally efficient random graph model capable of generating synthetic networks from a known NDD. By preserving the expected degree sequence, the

model captures fundamental structural characteristics of real-world graphs, making it particularly suitable for graph mimicking and for producing synthetic networks with high structural fidelity to empirical data.

To address the limitations of GenCAT, acMARK [7] allow the generation of graphs in normal and uniform distributions. The algorithm also make use of latent factors to generate topological relations and feature relations over the examples, however, it allow only homophily graphs, which are more suitable for community detection tasks. The graph generation is evaluated using benchmarks for community detection over different topologies created using different values of parameters.

Although homophily graphs are more suitable for clustering, real-world networks typically has heterophilic characteristics. AL-BTER allow de generation of graphs with low assortativity, which represents more challenge on graph learning tasks. Besides low node assortativity, the model allows the generation of low attribute assortativity, i.e, similar attributed nodes (measured by cosine similarity) tends to have different classes. To guarantee the quality of the proposed approach, AL-BTER was evaluated on synthetic graph proprieties and tasks such as classification using GNNs.

SkyMap [10] is a generative model designed to create synthetic attributed and labeled graphs for benchmarking GNNs. Unlike earlier models such as ALBTER and GenCAT, which struggle to match the learnability characteristics of real datasets, SkyMap was specifically built to reproduce GNN performance with high fidelity across architectures. To achieve this, the model takes as input a set of graph metrics (such as covering class distributions, mixing patterns, and feature correlations), and uses them to guide a four-phase generation pipeline: generating subgraphs per class, combining them using a mixing matrix, and assigning node features via a class-feature matrix. This architecture allows SkyMap to capture both topological and semantic properties of real-world graphs, resulting in significantly lower Wasserstein distances in accuracy distributions between synthetic and original datasets. The model was evaluated using Wassertain distance in order to show that Skymap is capable to recreate more accurately real-world graphs.

3. Synthetic Community-Aware Attributed Graph Generator

To address datasets exhibiting heterogeneous densities, to enable the control of specific node degree distributions and to create a graph with

sub-community structures, we propose the *Synthetic Community-Aware Attributed Graph Generator for community detection benchmarking via node degree distribution* (SCAtt). The SCAtt framework is designed to support the three tasks introduced in Section 2.1, namely: (1) the generation of a fully synthetic graph based solely on user-defined parameters; (2) the cloning of an existing graph while preserving its structural properties; and (3) the augmentation of a cloned graph through controlled modifications.

Regardless of the selected task, the SCAtt algorithm follows a unified pipeline composed of four sequential stages: (i) node community and sub-community assignment, (ii) edge construction, (iii) topological noise generation, and (iv) attribute assignment. Each stage plays a distinct role in ensuring that the resulting graph faithfully reflects both the community structure and the desired topological and attribute-related properties. The details of each stage are described in the following subsections.

3.1. SCAtt generation from user-defined inputs

In the user-driven generation mode, the graph is fully specified through a collection of matrices, constants, and probability distributions provided as input. These parameters collectively describe the target community structure, intra- and inter-community connectivity patterns, and node degree characteristics. Table 1 summarizes the complete set of inputs required for the construction of the output graph using the SCAtt framework.

3.1.1. Node community and sub-community assignment stage

Let

$$\mathbf{y} = [y_1, y_2, \dots, y_k]$$

denote the vector specifying the number of nodes assigned to each of the k communities, where y_c corresponds to the cardinality of community c . Based on this information, the algorithm constructs a community label vector

$$\mathbf{Y} \in \mathbb{R}^n,$$

with $n = \sum_{c=1}^k y_c$, such that each entry Y_i indicates the community label associated with node i . This procedure ensures that every node is deterministically assigned to exactly one community, fully respecting the predefined community sizes.

In addition to community-level assignments, SCAtt introduces a finer-grained structure by defining sub-communities within each community. For

Parameter	Type	Description
n	Integer	Total number of nodes in the graph.
k	Integer	Number of communities (or classes) in the graph.
d	Integer	Dimension of feature matrix X
$\mathbf{y}$	Vector	Vector of positive integers specifying the number of nodes in each class, with $\dim(\mathbf{y}) = k$.
$\mathbf{e}$	Vector	Vector of positive integers representing the number of homogeneous (intra-class) edges in each class, with $\dim(\mathbf{e}) = k$.
$\mathbf{dst}$	Vector	Vector of node degree distributions, where each element defines the node degree distribution associated with a class, with $\dim(\mathbf{dst}) = k$.
$\mathbf{M}_c$	Vector	Set of vectors, where each vector specifies the number of nodes assigned to each sub-community within a community.
ρ	Real	Target edge density of the output graph.
C	Matrix	Set of matrices defining the connection probabilities between sub-communities within each community.
N	Matrix	Matrix encoding the number of edges allocated between communities.

Table 1: Input parameters of the SCAtt graph generation algorithm.

a given community c , the vector $\mathbf{M}_c$ specifies the distribution of its sub-communities; that is, the i -th entry of $\mathbf{M}_c$ represents the probability that a node in community c belongs to sub-community i . Based on this distribution, the algorithm assigns each node in community c to exactly one of its corresponding sub-communities.

At the end of this stage, the algorithm possesses two fundamental pieces of information required for subsequent steps: each node is uniquely associated with both a community and a sub-community. This joint assignment provides the structural foundation for edge generation, enabling the controlled modeling of intra and inter sub-community connectivity patterns in the following stages.

3.1.2. Edge Construction Stage

In the edge construction stage, each class is assigned a predefined number of *homogeneous edges*, i.e., edges connecting nodes belonging to the same community, as specified by the vector $\mathbf{e}$. This design choice enables explicit control over the intra-community connectivity, making the resulting graph particularly suitable for community detection tasks.

In addition, each community is associated with a degree-generating distribution that governs the node degree distribution within that community. By controlling this distribution, the algorithm allows the formation of topologies with distinct structural roles, such as core and border nodes. The model supports three degree distributions: *normal*, *power-law*, or *uniform*.

For the normal distribution, the mean μ and standard deviation σ are defined by default as

$$\mu = \frac{n+1}{2}, \quad \sigma = \max\left(\frac{n}{6}, 1.0\right), \quad (1)$$

where n denotes the number of nodes in the corresponding sub-community. The probability density function is given by

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right). \quad (2)$$

For the power-law distribution, the degree probability follows

$$f(x) = x^{-\alpha}, \quad (3)$$

with the default exponent set to $\alpha = 2$, enabling the emergence of hub-dominated structures.

For the uniform distribution, all nodes are sampled with equal probability:

$$f(x) = \frac{1}{n}. \quad (4)$$

For a given community c , the source and target partitions used to generate an edge are sampled according to the sub-community interaction matrix C_c . Subsequently, the selection of the source and target nodes within their respective partitions is guided by the class-specific degree distribution $\mathbf{d}_c$. Since these distributions are constructed independently for each partition, the algorithm preserves local structural properties while maintaining global

consistency. To ensure that all nodes remain eligible for selection, a small constant value of 1×10^{-6} is added to each probability.

This strategy enables the generation of multiple sub-communities sharing the same degree distribution while emphasizing local topological characteristics, such as centrality and connectivity heterogeneity.

3.1.3. Noise Generation Stage

At this stage, the graph consists exclusively of homogeneous edges and may remain disconnected. To reflect realistic network scenarios and to support node clustering benchmarks, the algorithm introduces structural noise in the form of heterogeneous edges, i.e., edges connecting nodes from different classes. The amount of noise is controlled through a user-defined density parameter $\rho \in \mathbb{R}$, which determines the number of heterogeneous edges to be added based on the class interaction matrix $\mathbf{N}_{k \times k}$, where $N_{i,j}$ represents the probability of connection between the communities i and j .

This design provides the user with two complementary mechanisms to control the graph topology: (1) by specifying the internal structure through community sizes, sub-communities, and degree distributions; and (2) by adjusting the global density parameter ρ , which regulates the level of inter-community connectivity. Together, these mechanisms allow fine-grained control over both intra-community organization and inter-community heterogeneity.

3.1.4. Attribute Generation Stage

After the structural construction of the graph, node attributes are generated with a community-conditional and topology-aware behavior. This stage aims to produce feature representations that simultaneously preserve community separability and incorporate structural information derived from the graph topology.

Let $\tilde{G} = (\tilde{V}, \tilde{E})$ be the graph obtained after the edge construction and noise generation stages, and let $\tilde{Y} \in \{1, \dots, k\}^{|\tilde{V}|}$ denote the corresponding class label vector. The attribute matrix $\tilde{X} \in \mathbb{R}^{|\tilde{V}| \times d}$ is generated through a three-step process: (i) construction of class prototype vectors, (ii) injection of stochastic variability, and (iii) topology-aware feature smoothing.

Community-Conditional Prototype Construction. Initially, each class is associated with a latent prototype vector in the attribute space. To ensure

linear independence and avoid redundant feature directions across communities, an orthonormal basis is constructed via QR decomposition. Specifically, a random matrix

$$\mathcal{A} \in \mathbb{R}^{d \times k}$$

is sampled from a standard normal distribution (where $k \leq d$), and its QR decomposition yields an orthonormal matrix $\mathbf{Q}$. The resulting community prototypes are defined as the rows of $\mathbf{Q}^\top$, such that each community c is assigned a prototype vector $\mathbf{q}_c \in \mathbb{R}^d$.

Stochastic Attribute Initialization. For each node $v \in \tilde{V}$ with community label $\tilde{y}_v = c$, the initial attribute vector $\mathbf{x}_v^{(0)}$ is generated as

$$\mathbf{x}_v^{(0)} = \mathbf{q}_c + \lambda \boldsymbol{\epsilon}_v, \quad (5)$$

where $\boldsymbol{\epsilon}_v \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$ is a Gaussian noise vector and $\lambda \in [0, 1]$ controls the intra-class dispersion. This formulation ensures that nodes belonging to the same class are centered around a common prototype while maintaining sufficient variability to avoid degenerate feature distributions.

Degree-Normalized Topological Smoothing. To incorporate structural information into the attribute space, a single-step propagation mechanism inspired by graph signal processing is applied. Let $\mathbf{A}$ denote the adjacency matrix of $\tilde{G}$ and let $\mathbf{D}$ be a diagonal matrix defined as

$$\mathbf{D}_{vv} = \begin{cases} \deg(v)^{-1/2}, & \text{if } \deg(v) > 0, \\ 0, & \text{otherwise.} \end{cases} \quad (6)$$

The final attribute matrix $\tilde{X}$ is obtained via the convex combination

$$\tilde{X} = \alpha \tilde{X}^{(0)} + (1 - \alpha) \mathbf{D} \mathbf{A} \tilde{X}^{(0)}, \quad (7)$$

where $\alpha \in [0, 1]$ controls the trade-off between community-conditional feature purity and neighborhood-induced smoothing.

The proposed formulation provides explicit control over the statistical and structural properties of the generated attributes through the parameters λ and α . The parameter λ regulates the variance of the Gaussian perturbation applied during the attribute initialization, directly controlling the intra-community dispersion in the feature space. Smaller values of λ yield more compact and well-separated clusters, while larger values increase

Algorithm 1 SCAtt over user-inputs

Require: Number of nodes n ; community sizes $\mathbf{y}$; number of communities k ; intra-community edge counts $\mathbf{e}$; partition interaction matrices $\{\mathbf{C}_c\}$; partition sizes $\{\mathbf{M}_c\}$; degree distribution types $\mathbf{d}$; target density ρ ; inter-community probability matrix $\mathbf{N}$; attribute parameters (d, λ, α) .

Ensure: Attributed graph $\tilde{G} = (\tilde{V}, \tilde{E}, \tilde{X}, \tilde{Y})$.

Initialize graph $\tilde{G}$ with n nodes and empty edge set

Assign community labels $\tilde{Y}$ according to $\mathbf{y}$

Assign nodes of each community to sub-communities based on $\{\mathbf{M}_c\}$

Homogeneous Edge Construction

for each class $c = 1, \dots, k$ **do**

 Construct intra-class edges using degree-based sampling

 Sample source and target partitions according to $\mathbf{C}_c$

end for

Noise Generation

Add heterogeneous edges between different classes until density ρ is reached

Reindex graph to remove isolated nodes

Attribute Generation

Generate orthogonal class prototype vectors in $\mathbb{R}^d$

Initialize node attributes with class prototypes and Gaussian noise (λ)

Apply degree-normalized topological smoothing weighted by α

return $\tilde{G}$

overlap between communities in the feature space, allowing the generation of more challenging scenarios for downstream learning tasks.

Conversely, the parameter α governs the relative contribution of community conditional information and topological smoothing in the final attribute representation. Higher values of α emphasize community prototypes and preserve feature purity, whereas lower values increase the influence of neighborhood aggregation, strengthening the coupling between node attributes and graph structure. Together, these parameters enable fine-grained adjustment of the attribute generation process, allowing the user to tailor the synthetic data to different levels of structural dependency and classification difficulty. Algorithm 1 describes the workflow of the proposed algorithm.

3.2. SCAtt for graph mimicking and augmentation

Let $G = (V, E, X, Y)$ be an attributed graph, where V is the set of nodes, E is the set of edges, $X \in \mathbb{R}^{|V| \times d}$ the node attributed matrix, and $Y \in \{1, \dots, C\}^{|V|}$ the community (or class) labels associated with each node. Additionally, assume that G can be decomposed into community-induced subgraphs $G_c = (V_c, E_c)$, where $V_c = \{v \in V : y_v = c\}$. Nodes in each community-induced subgraphs are homogeneous, i.e., the set of edges E_c connect elements that belong to the same community c .

For the graph mimicking and augmentation task, the objective of the proposed algorithm is to generate a new attributed graph $\tilde{G} = (\tilde{V}, \tilde{E}, \tilde{X}, \tilde{Y})$ that preserves the structural, community-level and connectivity homogeneity patterns observed in G , while optionally allowing the graph to be augmented to a larger number of nodes, based on node degree distributions of the original data.

The way that the task will be concluded is similar to the graph generation via user-inputs task. So, in next sections we describe the main differences between the tasks.

3.2.1. Node community Assignment and sub-community Discovery Stage

A new undirected and unweighted graph $\tilde{G}$ is initialized with the same number of nodes as the original graph G . The node label vector $\tilde{Y}$ is directly inherited from Y , ensuring community consistency at the node level.

To capture the topological organization within each sub-community in G_c , a community detection algorithm, Parallel Louvain Method (PLM) by default, is applied to identify a set of sub-communities

$$\mathcal{P}_c = \{P_{c,1}, \dots, P_{c,K_c}\}.$$

Based on this partitioning, a sub-community interaction matrix $\mathbf{C}_c \in \mathbb{R}^{K_c \times K_c}$ is computed for each class, where each entry quantifies the empirical tendency of nodes in sub-community $P_{c,i}$ to establish connections with nodes in sub-community $P_{c,j}$.

3.2.2. Edge Construction and Noise Stages

After identifying the partitions associated with each class, the edge construction stage begins. The source and target partitions for every edge in the synthetic graph $\tilde{G}$ is guided by the relative sizes of the corresponding partitions in the original graph G . Subsequently, the selection of individual

nodes within the chosen partitions is performed according to the degree distribution observed in G . This strategy ensures that both local and global topological characteristics are preserved in the replicated graph structure.

At this step of the algorithm, the graph is composed only of homogeneous edges. From G it is possible to extract a matrix $N_{k \times k}$ where $N_{i,j}$ is the probability of a heterogeneous edge connect the classes i and j . Thus, heterogeneous edges are added to $\tilde{G}$ until the density ρ of G is reached on $\tilde{G}$.

At this point of the algorithm, the topological structure is complete and it is based on the original graph. The absence of external distribution as in GenCAT [4] and SkyMAP [10] allow the generated graph to be faithful to the original one.

3.2.3. Attribute Assignment Stage

This stage aims to generate the attribute matrix $\tilde{X}$ of the mimicked graph $\tilde{G}$ by directly leveraging the node features of the original graph G . The underlying assumption is that nodes occupying similar structural roles within the same community should exhibit comparable attribute representations. To this end, a rank-based matching strategy is employed to transfer node attributes from G to $\tilde{G}$ in a label-conditional and topology-aware manner.

The attribute generation process is performed independently for each community label $c \in \{1, \dots, k\}$. For a given community c , we first identify the subset of nodes $V_c \subset V$ such that $y_v = c$, and consider the corresponding nodes $\tilde{V}_c \subset \tilde{V}$ in the mimicked graph.

Within each class, nodes in both graphs are ordered according to their degree, resulting in two degree-ranked sequences: one derived from the original graph G and another from the mimicked graph $\tilde{G}$. A one-to-one correspondence between nodes is then established by matching nodes with identical rank positions in these ordered lists. In this way, nodes with similar degree-based structural importance in G are aligned with nodes playing analogous roles in $\tilde{G}$.

Once this correspondence is defined, the attribute vector of each node in the mimicked graph is initialized by copying the attribute vector of its matched counterpart in the original graph. To avoid exact duplication and to introduce controlled variability, an additive noise term sampled from a Gaussian distribution is applied to each transferred feature vector. Formally, for a matched node pair $(v, \tilde{v})$, the attribute assignment is given by

$$\tilde{\mathbf{x}}_{\tilde{v}} = \mathbf{x}_v \quad (8)$$

This rank-based attribute matching strategy preserves the empirical distribution of node features within each class while ensuring consistency with the reconstructed topology. By conditioning the transfer on both class membership and degree-based structural roles, the method maintains semantic coherence between structure and attributes, making the resulting synthetic graph suitable for downstream tasks such as clustering, node classification and graph representation learning.

3.3. Graph Augmentation Stage

In addition to mimicking an existing attributed graph, the proposed framework supports *data augmentation* by increasing the size of the synthetic graph to a user-defined number of nodes. Given a target size $|\tilde{V}| = n'$, with $n' > |V|$, the algorithm adds $n' - |V|$ new nodes to the mimicked graph $\tilde{G}$ while preserving (i) the empirical community proportions, (ii) the sub-community structure within each label, and (iii) the degree-driven connectivity patterns observed in the reconstructed topology.

Partition model for the augmented graph. Before inserting new nodes, the algorithm recomputes the partition structure within each community induced-subgraph of $\tilde{G}$ using the same community detector employed in the mimic stage. For each community c , this yields a set of sub-communities $\mathcal{P}_c = \{P_{c,1}, \dots, P_{c,K_c}\}$ and a corresponding sub-community interaction matrix $\mathbf{C}_c \in \mathbb{R}^{K_c \times K_c}$, where $C_{i,j}$ is the portion of edges that connect the sub-communities i and j , which summarizes the empirical tendency of edges to occur between sub-communities. This partition model is then used as a guide for placing and wiring new nodes in a topology-consistent way.

Community and sub-community assignment. Each new node v_i is first assigned to a community c_i by sampling from the empirical community distribution observed in $\tilde{Y}$, i.e., communities with larger support in the mimicked graph are more likely to receive additional nodes. Conditioned on the sampled community, the algorithm assigns v_i to a source partition $P_{c_i,s}$ by sampling from the partition probability distribution of that community (which is proportional to partition sizes). This step ensures that the augmented graph preserves both the global class balance and the relative sizes of partitions inside each class.

Attribute initialization for new nodes. After assign the node sub-community, the attribute vector of the new node is initialized to be representative of the local attribute distribution of its sub-community. Concretely, the feature vector $\tilde{\mathbf{x}}_{v_i}$ is set to the mean of the feature vectors of nodes currently belonging to $P_{c_i,s}$:

$$\tilde{\mathbf{x}}_{v_i} \leftarrow \frac{1}{|P_{c_i,s}|} \sum_{v \in P_{c_i,s}} \tilde{\mathbf{x}}_v.$$

This strategy preserves community-level attribute coherence, ensuring that added nodes remain compatible with the attribute geometry learned during the mimic stage.

Edge formation via sub-community interactions. To incorporate the new node into the graph structure, the algorithm creates a number of edges that reflects the typical connectivity of the selected partition. Specifically, the number of connections for v_i is set to the average degree of homogeneous nodes in the sub-community $P_{c_i,s}$, approximated by

$$m_i \approx \frac{2|E(P_{c_i,s})|}{|P_{c_i,s}|},$$

with $m_i \geq 1$ to avoid isolating the new node.

For each of the m_i edges, a target sub-community $P_{c_i,t}$ is sampled according to the interaction profile of the source sub-community, given by the row $\mathbf{C}_c[s, :]$. Finally, a target node $v_j \in P_{c_i,t}$ is sampled with probability proportional to its current degree, promoting preferential attachment and preserving the degree heterogeneity of the reconstructed graph. The edge (v_i, v_j) is added to $\tilde{E}$, and the interaction matrix is updated to reflect the newly created inter-partition connection.

Overall, the augmentation stage extends the mimicked graph without disrupting its learned organization. By sampling communities and sub-community according to their empirical prevalence, initializing attributes from sub-community statistics, and wiring new nodes through sub-community level interaction, the procedure yields larger synthetic graphs that remain structurally and semantically consistent with the original network.

4. Experiments and Experimental Setup

5. Experiments and Experimental Setup

In this section, we describe the experimental configuration adopted to evaluate the proposed model. We detail the methodological steps designed to analyze the behavior of SCAtt and to assess its performance in comparison with state-of-the-art attributed graph generators. The evaluation includes controlled parametric analysis, clustering benchmarks, and structural property validation.

5.1. Ablation Study Over Noise Parameters

To evaluate the parameters λ, α (Equations 5, 7) and the density parameter ρ , we conduct an experiment where we define a baseline model that consists in a Graph Auto-Encoder [14] to learn node representations and KMeans (we call it GAE+KMEANS) algorithm do cluster the data based on the number of generated communities.

The GAE+KMEANS baseline allow us to understand the behavior of the hyperparameters that controls the noise addition to the model. The GAE architecture contains 3 layer with 60, 32 and 16 neurons respectively for the encoder. The latent representation at the layer with 16 dimensions is used to feedforward the KMeans algorithm. We evaluate the results based on Normalized Mutual Information (NMI), comparing the ground-truth generated by the model and the obtained by GAE+KMEANS.

For the graph construction, we set $n = 120$, $k = 3$, $d = 60$, $\mathbf{y} = [40, 40, 40]$, i.e, 40 nodes per class, $\mathbf{e} = [150, 200, 100]$, the sub-community interaction set of matrices

$$C = \left\{ [1], \begin{bmatrix} 0.6 & 0.3 & 0.1 \\ 0.1 & 0.7 & 0.2 \\ 0.0 & 0.3 & 0.7 \end{bmatrix}, \begin{bmatrix} 0.9 & 0.1 \\ 0.1 & 0.9 \end{bmatrix} \right\},$$

the sub-community probability set of vectors

$$M = \{[1], [0.4, 0.3, 0.3], [0.9, 0.1]\},$$

and the community noise matrix

$$N = \begin{bmatrix} 0 & 0.2 & 0.8 \\ 0.3 & 0 & 0.7 \\ 0.5 & 0.5 & 0 \end{bmatrix}.$$

This input parameters for SCAtt are suitable to generate graphs that can be easily used to evaluate noise parameters. To guarantee reproducibility we set a seed to solely evaluate the parameters. We evaluate ρ at the values $[0.03, 0.08, 0.15, 0.3, 0.5]$ and α and λ at the interval $[0,1.0]$ with step 0.2.

Figure 1 illustrate the results obtained by the experiments. Originally the network has 450 homogeneous edges, this mean that a ρ value equals to 0.03 does not generate any heterogeneous edges, which alleviate the task of clustering. With larger values of ρ , the classification task turns way harder, as expected.

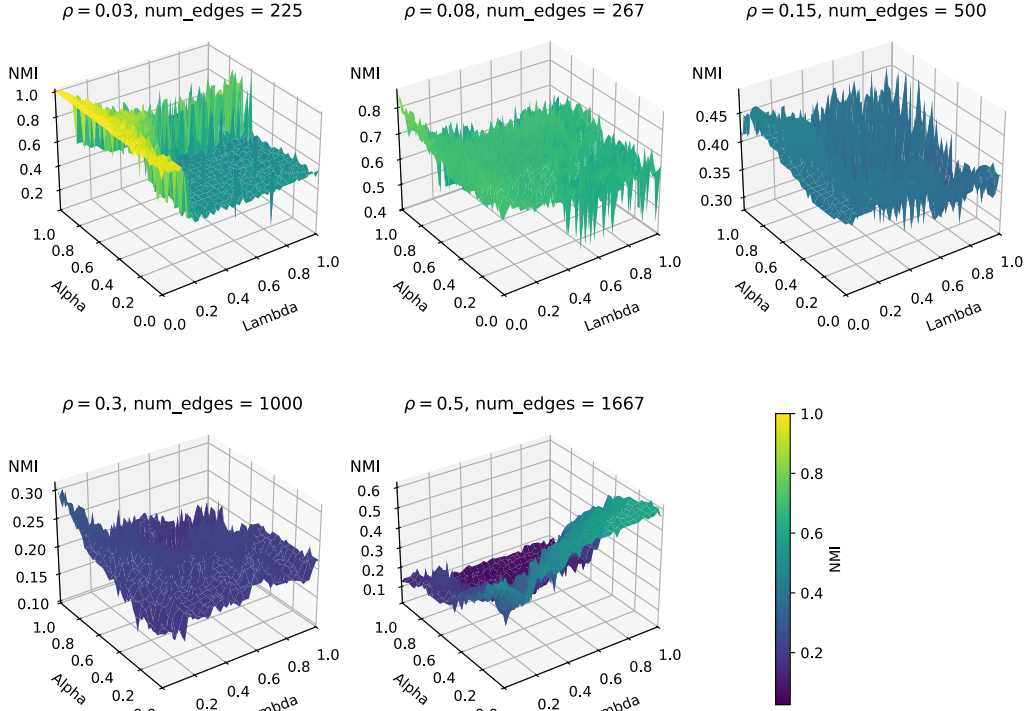


Figure 1: NMI analysis over diferent values of ρ, α and λ .

The highest the value of λ is, the lowest is the NMI score, since this parameter is responsible for add Gaussian Noise into the features. The value of α otherwise, contributes to the semantic transmission of the graph, i.e, higher values of alpha create features that are more similar between the clusters. Although, when the number of heterogeneous edges are higher (higher values of ρ), higher values of α indicate a lower NMI score, since

semantic information is traded with other clusters.

5.2. Parametric Evaluation over Clustering Algorithms

To demonstrate the effectiveness of SCAtt as a graph generator for controlled parametric evaluation of clustering algorithms, we selected DGCluster [15], a semi-supervised node clustering algorithm capable of simultaneously inducing the number of clusters (k_{est}) to perform the clustering task. DGCluster is formulated as the combination of two loss components, L_1 and L_2 , derived respectively from global cluster quality metrics and local information consistency. These components are balanced through a trade-off parameter $\lambda_{\text{DGCluster}}$, which controls the relative importance of the local information loss in the optimization process.

Using SCAtt, we generate graph instances with controlled topological and semantic properties, allowing us to systematically analyze the sensitivity of DGCluster to variations in $\lambda_{\text{DGCluster}}$ and in the number of generated communities. Performance is evaluated using the Normalized Mutual Information (NMI) metric.

To evaluate the sensitivity of the algorithm with respect to the number of generated communities, we used SCAtt to construct 18 networks with approximately 13,000 vertices each, systematically varying the number of communities. This procedure allows us to analyze the behavior of DGCluster under different structural granularities, that is, with varying numbers of groups and levels of community fragmentation.

The graphs were generated using heuristics that define both the total number of vertices and the internal distribution of each community. Further details about the generation process can be found in [link](#). Figure 2 summarizes the results obtained in this experiment.

For graphs with a smaller number of communities, DGCluster demonstrated a greater ability to correctly identify the number of groups defined by SCAtt. However, the NMI values were relatively lower in these scenarios. As the number of communities increases, DGCluster shows greater difficulty in accurately estimating the number of clusters. On the other hand, higher NMI values tend to be observed.

This behavior indicates that higher NMI scores are associated with scenarios involving a larger number of groups, a result that can also be observed in the experiments conducted by the authors [15].

To evaluate the performance of the trained model under different values of $\lambda_{\text{DGCluster}}$, we generated four networks in five seeds using the same parameters

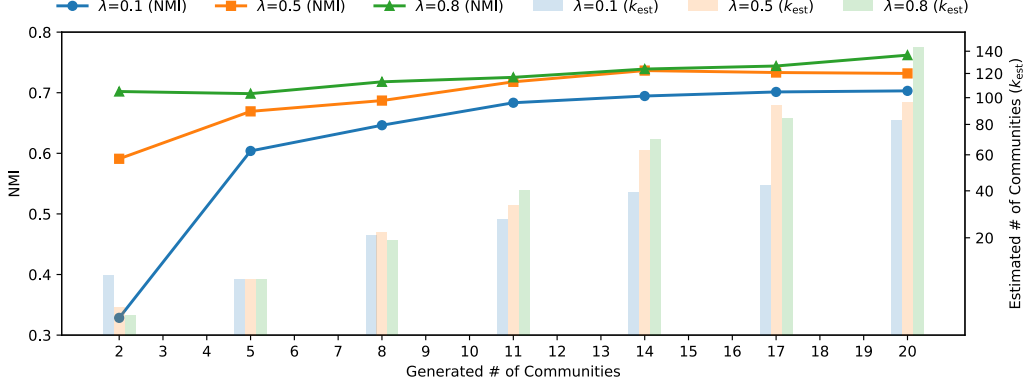


Figure 2: DGCluster benchmark using SCAtt for different values of λ .

adopted in the parametric test (Section 5.1). However, we varied the values of ρ within the set $0.08, 0.10, 0.14, 0.20$ and $\lambda_{\text{DGCluster}}$ within the interval $[0, 1.0]$ with a step size of 0.1 . Figure 5.2 summarizes the results of these experiments.

A consistent increase in NMI can be observed as $\lambda_{\text{DGCluster}}$ increases in scenarios where the graph contains less noise, that is, lower values of ρ . This behavior is similar to what was reported for real-world datasets in the original DGCluster study. However, as the dataset becomes more challenging, the monotonic growth pattern of $\lambda_{\text{DGCluster}}$ no longer holds. This suggests that the effectiveness of $\lambda_{\text{DGCluster}}$ is influenced by higher levels of heterophily in the data.

5.3. Scale-Free Analysis

In order to evaluate the preservation of topological properties during the graph mimicking process, which involves data cloning and an increase in the number of vertices, we apply the scale-free plausibility test proposed by [12]. After assessing the plausibility of the power-law hypothesis, each network is classified according to its degree of adherence to this distribution, following an approach similar to that adopted in [11]. The objective of this test is to evaluate whether the node degree distribution can be plausibly described by a power-law.

Let G be a graph whose degree sequence $\{x_i\} = x_1, x_2, \dots, x_k$ is modeled by a power-law distribution of the form

$$\Pr(x) = Cx^{-\alpha}, \quad \alpha > 1, \quad x \geq x_{\min} \geq 1,$$

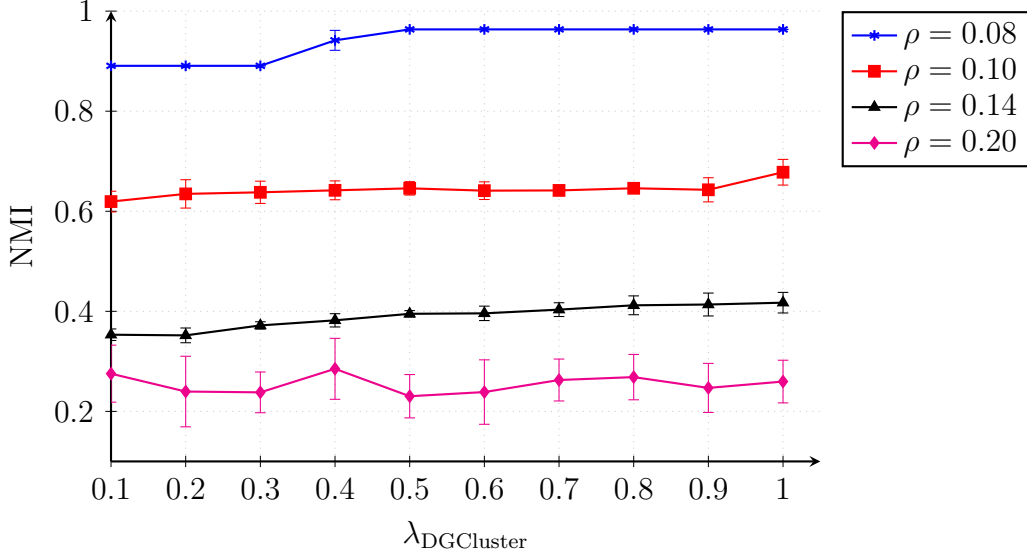


Figure 3: NMI obtained over $\lambda_{\text{DGCluster}}$ for different values of ρ .

where α denotes the scaling exponent, C is the normalization constant, and x represents the integer-valued vertex degrees. This modeling is applied exclusively to the upper tail of the distribution, defined by the set of values $x \geq x_{\min}$.

To identify the power-law distribution that best describes the degree distribution of graph G , it is necessary to determine an optimal value of $x_{\min}$ that marks the beginning of the upper tail, as well as to estimate the corresponding value of the exponent α using only the truncated data.

The procedure begins by evaluating each candidate value of $x_{\min}$. For each candidate, the exponent $\hat{\alpha}$ is estimated via maximum likelihood for the discrete distribution according to

$$\hat{\alpha} = 1 + n \left[\sum_{i=1}^n \ln \left(\frac{x_i}{x_{\min} - \frac{1}{2}} \right) \right]^{-1}, \quad (9)$$

where x_i , $i = 1, \dots, n$, are the observed values such that $x_i \geq x_{\min}$. The estimated value of $\hat{\alpha}$ is then used to assess how well a power-law explains the tail of the distribution.

Let $S(x)$ denote the empirical cumulative distribution function (CDF) of the tail data, and let $P(x)$ denote the theoretical CDF of a power-law

with parameters $\hat{\alpha}$ and $x_{\min}$. The selection of $x_{\min}$ is performed using the Kolmogorov-Smirnov statistic, defined as

$$D(x_{\min}) = \max_{x \geq x_{\min}} |S(x) - P(x)|. \quad (10)$$

The optimal value $\hat{x}_{\min}$ is obtained by minimizing the statistic $D(x_{\min})$, that is,

$$\hat{x}_{\min} = \arg \min_{x_{\min}} D(x_{\min}).$$

However, minimizing the Kolmogorov-Smirnov statistic alone does not guarantee that the degree distribution is adequately described by a power-law. To assess the plausibility of the fit, a goodness-of-fit test based on Monte Carlo simulations is applied. In this test, 1000 synthetic distributions are generated from power-law models whose parameters are estimated from the observed data.

For each synthetic replicate, the parameters $x_{\min}$ and $\hat{\alpha}$ are re-estimated, and the corresponding value of the statistic D (Equation 10) is computed. The p -value is defined as the fraction of replicates for which the value of D is greater than or equal to the value observed in the real data. Values of $p > 0.1$ indicate that the power-law hypothesis is plausible for the analyzed distribution.

Then, the distribution that better fit the data is compared to alternative distributions via likelihood test.

In order to evaluate de different graphs generated over different numbers of edges, we use an approach similar to [11], where we create a ton of rules that guide the classification of every class in the graph that we want to evaluate the power-law plausibility. Each subgraph is classified as “Not Scale-Free”, “Weak Scale-Free” and “Strong Scale-Free”.

- **Strong Scale-Free** regard over networks that have a p -value greater than 0.1, the estimated value of $\hat{\alpha}$ is between 2.0 and 3.0, and no alternative distribution is favored.
- **Weak Scale-Free** regard over networks that also have a p -value greater than 0.1, and no other alternative is favored, but the value of $\hat{\alpha}$ is not between the interval of $[2.0, 3.0]$
- **Not Scale-Free** networks are the ones that do have the p -value greater than 0.1 or some alternative is favored.

We compare the proposed approach with GenCAT [4] and Chung–Lu [13] on the CiteSeer and PubMed datasets. Tables 2 and 3 present the classification results for the cloned datasets at their original size and when augmented by $1.3\times$ and $1.5\times$. Subgraphs highlighted in bold indicate those classified into the same category as the original dataset.

α			n_{tail}			D			Class			Subgraph	Model
$1\times$	$1.3\times$	$1.5\times$	$1\times$	$1.3\times$	$1.5\times$	$1\times$	$1.3\times$	$1.5\times$	$1\times$	$1.3\times$	$1.5\times$		
2.19	2.11	2.13	247	547	651	0.02	0.03	0.03	Strong	Strong	Strong	0	GenCAT
2.27	2.10	2.00	1058	1758	3609	0.02	0.01	0.02	Strong	Strong	Strong	1	GenCAT
2.26	2.08	2.01	739	1754	4051	0.01	0.01	0.02	Strong	Strong	Strong	2	GenCAT
2.21	-	-	1187	-	-	0.05	-	-	Strong	-	-	0	chung-lu
3.14	-	-	401	-	-	0.04	-	-	Weak	-	-	1	chung-lu
4.14	-	-	233	-	-	0.03	-	-	Weak	-	-	2	chung-lu
2.22	2.56	2.61	1215	1529	1961	0.05	0.02	0.02	Strong	Strong	Strong	0	SCAtt
3.21	2.39	2.55	398	3925	3675	0.03	0.02	0.02	Weak	Strong	Strong	1	SCAtt
4.17	4.29	2.42	195	161	4021	0.03	0.04	0.03	Weak	Weak	Strong	2	SCAtt
2.03	-	-	1629	-	-	0.05	-	-	Strong	-	-	0	PubMed
3.11	-	-	455	-	-	0.03	-	-	Weak	-	-	1	PubMed
4.22	-	-	137	-	-	0.03	-	-	Weak	-	-	2	PubMed

Table 2: Scale-Free Classification of PubMed dataset.

For the PubMed dataset, the proposed model achieved significant results, with a loss in classification consistency occurring only when the number of vertices was increased by $1.5\times$. In comparison to GenCAT, the node degree distribution remained more stable under augmentation. The Chung–Lu algorithm also produced relevant results, however, this model is not inherently designed to increase the number of vertices in the cloned graph.

For the CiteSeer dataset, the best results were obtained when the number of vertices was increased by $1.5\times$. In the smaller augmentation setting, SCAtt was not able to fully capture the scale-free characteristics of certain subgraphs. Nevertheless, when considering all three experimental scenarios jointly, the proposed approach outperformed the baseline algorithms overall.

6. Discussion and Conclusion

This paper introduced SCAtt, a synthetic community-aware attributed graph generator designed for benchmarking community detection and graph learning algorithms. Unlike existing generators that impose restrictive structural assumptions, SCAtt enables explicit control over community sizes, sub-community structures, node degree distributions, inter-community noise, and attribute generation mechanisms. This design allows the creation of graphs

α			n_{tail}			D			Class			Subgraph	Model
1 $\times$	1.3 $\times$	1.5 $\times$	1 $\times$	1.3 $\times$	1.5 $\times$	1 $\times$	1.3 $\times$	1.5 $\times$	1 $\times$	1.3 $\times$	1.5 $\times$		
2.21	2.44	2.50	12	8	15	0.01	0.12	0.07	Weak	Weak	Weak	0	GenCAT
2.71	2.63	2.34	73	52	81	0.04	0.04	0.05	Strong	Strong	Strong	1	GenCAT
2.46	2.52	2.86	365	68	22	0.04	0.03	0.05	Strong	Strong	Weak	2	GenCAT
2.34	2.39	2.62	104	89	107	0.02	0.03	0.03	Strong	Strong	Strong	3	GenCAT
2.49	2.78	2.36	145	41	155	0.03	0.06	0.04	Strong	Weak	Strong	4	GenCAT
2.21	2.36	2.48	76	56	209	0.05	0.03	0.05	Strong	Strong	Strong	5	GenCAT
3.54	-	-	26	-	-	0.05	-	-	Weak	-	-	0	chung-lu
2.90	-	-	124	-	-	0.05	-	-	Strong	-	-	1	chung-lu
3.53	-	-	44	-	-	0.03	-	-	Weak	-	-	2	chung-lu
2.90	-	-	189	-	-	0.05	-	-	Strong	-	-	3	chung-lu
2.99	-	-	124	-	-	0.03	-	-	Strong	-	-	4	chung-lu
7.55	-	-	7	-	-	0.05	-	-	Weak	-	-	5	chung-lu
3.97	3.22	3.37	24	28	21	0.03	0.04	0.02	Weak	Weak	Weak	0	SCAtt
3.83	2.68	2.32	54	171	193	0.06	0.04	0.06	Weak	Strong	Strong	1	SCAtt
3.44	2.56	2.65	57	294	143	0.04	0.04	0.04	Weak	Strong	Strong	2	SCAtt
4.09	2.75	3.36	50	254	114	0.06	0.04	0.04	Weak	Strong	Weak	3	SCAtt
3.93	3.19	3.23	61	94	83	0.05	0.03	0.02	Weak	Weak	Weak	4	SCAtt
4.01	2.81	3.94	44	163	48	0.05	0.04	0.04	Weak	Strong	Weak	5	SCAtt
2.70	-	-	39	-	-	0.05	-	-	Weak	-	-	0	CiteSeer
2.55	-	-	193	-	-	0.05	-	-	Strong	-	-	1	CiteSeer
3.00	-	-	84	-	-	0.04	-	-	Strong	-	-	2	CiteSeer
5.98	-	-	23	-	-	0.05	-	-	Weak	-	-	3	CiteSeer
2.89	-	-	187	-	-	0.03	-	-	Strong	-	-	4	CiteSeer
3.72	-	-	54	-	-	0.04	-	-	Weak	-	-	5	CiteSeer

Table 3: Scale-Free Classification of CiteSeer dataset.

with adjustable structural and semantic complexity, supporting systematic evaluation under controlled conditions.

The experimental analysis demonstrates that SCAtt enables interpretable manipulation of structural heterogeneity and attribute dispersion. By independently controlling ρ , λ , and α , the framework allows researchers to progressively increase task difficulty and study how clustering performance degrades under higher structural mixing and feature noise.

To illustrate this capability, DGCluster [15] was selected as a representative clustering method for parametric evaluation. However, the proposed framework is not restricted to this algorithm and can be used to benchmark any GNN-based or graph learning approach. The controlled experiments reveal that algorithm sensitivity to hyperparameters is strongly influenced by structural conditions, particularly under higher heterophily levels. This highlights the importance of synthetic generators that allow systematic stress-testing beyond fixed real-world datasets.

In the mimicking and augmentation scenarios, SCAtt preserves structural properties more consistently than baseline generators. Nevertheless, some limitations remain. The mimicking stage depends on an external com-

munity detection algorithm to recover sub-community structures, which introduces sensitivity to the chosen partitioning method. Additionally, the attribute transfer mechanism relies on rank-based matching between original and synthetic nodes. While this strategy preserves structural roles, it partially reduces feature-level generalization, since attributes are derived from the original data rather than being independently generated.

Overall, SCAtt provides a flexible and interpretable framework that bridges fully synthetic graph generation and structure-preserving dataset replication. By enabling fine-grained structural control while maintaining empirical plausibility, the model offers a practical tool for advancing research in attributed graph clustering and graph neural networks.

References

- [1] G. N. Chandrika, K. Alnowibet, K. S. Kautish, E. S. Reddy, A. F. Al-rasheedi, A. W. Mohamed, Graph transformer for communities detection in social networks, *Computers, Materials & Continua* 70 (3) (2022). doi:10.32604/cmc.2022.021186.
- [2] L. Zhuo, Y. Chen, B. Song, Y. Liu, Y. Su, A model for predicting ncna-protein interactions based on graph neural networks and community detection, *Methods* 207 (2022) 74–80. doi:10.1016/j.ymeth.2022.09.001.
- [3] X. Wei, Z. Zhang, H. Huang, Y. Zhou, An overview on deep clustering, *Neurocomputing* 590 (2024) 127761. doi:10.1016/j.neucom.2024.127761.
- [4] S. Maekawa, Y. Sasaki, G. Fletcher, M. Onizuka, Gencat: Generating attributed graphs with controlled relationships between classes, attributes, and topology, *Information Systems* 115 (2023) 102195. doi:https://doi.org/10.1016/j.is.2023.102195.
- [5] G. H. Messias, T. Gonçalves Botelho, S. Iasulaitis, M. Coelho Naldi, A. Demétrius Baria Valejo, A systematic review of graph neural network for node clustering in homogeneous attributed networks, *IEEE Access* 13 (2025) 215265–215285. doi:10.1109/ACCESS.2025.3646597.
- [6] J. Zhou, C. Xie, S. Gong, Z. Wen, X. Zhao, Q. Xuan, X. Yang, Data augmentation on graphs: A technical survey, *ACM Computing Surveys* 57 (11) (2025) 1–34.

- [7] S. Maekawa, J. Zhang, G. Fletcher, M. Onizuka, General generator for attributed graphs with community structure, in: Proceeding of the ECML/PKDD graph embedding and mining workshop, 2019, pp. 1–5.
- [8] P. Andreeva, K. Bochenina, E. Shikov, Attributed labeled bter-based generative model for benchmarking of graph neural networks (Sep. 2023). doi:10.21203/rs.3.rs-3288680/v1.
URL <http://dx.doi.org/10.21203/rs.3.rs-3288680/v1>
- [9] O. Benyahia, C. Largeron, B. Jeudy, O. R. Zaïane, DANCer: Dynamic Attributed Network with Community Structure Generator, Springer International Publishing, 2016, p. 41–44. doi:10.1007/978-3-319-46131-1_9.
- [10] A. Wassington, R. Higuera, S. Abadal, Skymap: a generative graph model for gnn benchmarking, Frontiers in Artificial Intelligence 7 (Nov. 2024). doi:10.3389/frai.2024.1427534.
URL <http://dx.doi.org/10.3389/frai.2024.1427534>
- [11] A. D. Broido, A. Clauset, Scale-free networks are rare, Nature Communications 10 (1) (Mar. 2019). doi:10.1038/s41467-019-08746-5.
URL <http://dx.doi.org/10.1038/s41467-019-08746-5>
- [12] A. Clauset, C. R. Shalizi, M. E. J. Newman, Power-law distributions in empirical data, SIAM Review 51 (4) (2009) 661–703. doi:10.1137/070710111.
URL <http://dx.doi.org/10.1137/070710111>
- [13] J. C. Miller, A. Hagberg, Efficient generation of networks with given expected degrees, in: International workshop on algorithms and models for the web-graph, Springer, 2011, pp. 115–126.
- [14] T. N. Kipf, M. Welling, Variational graph auto-encoders (2016). arXiv:1611.07308.
- [15] A. Bhowmick, M. Kosan, Z. Huang, A. Singh, S. Medya, Dgcluster: A neural framework for attributed graph clustering via modularity maximization, Proceedings of the AAAI Conference on Artificial Intelligence 38 (10) (2024) 11069–11077. doi:10.1609/aaai.v38i10.28983.